

SmolVLM

Redefining Small and Efficient Multimodal Models

Hugging Face & Stanford University

256M

Smallest Model

<1 GB

GPU RAM

59.8%

Avg Score (2.2B)

80 tok/s

On-Device

Large VLMs Are Powerful but Impractical for Edge Deployment

The gap between SOTA capability and deployable footprint

- ▶ **State-of-the-art VLMs require 27+ GB GPU memory** — Molmo-A1B-7B needs 27.7 GB, ruling out mobile/edge
- ▶ **Massive parameter counts dominate cost** — 7B+ parameters demand datacenter-grade hardware
- ▶ **On-device AI is growing rapidly** — privacy, latency, and connectivity demand local inference
- ▶ **Current efficient models sacrifice too much capability** — compression often destroys task accuracy

Molmo-A1B-7B

7B params

27.7 GB RAM

InternVL2-2B

2B params

~8 GB RAM

SmoVLM-256M

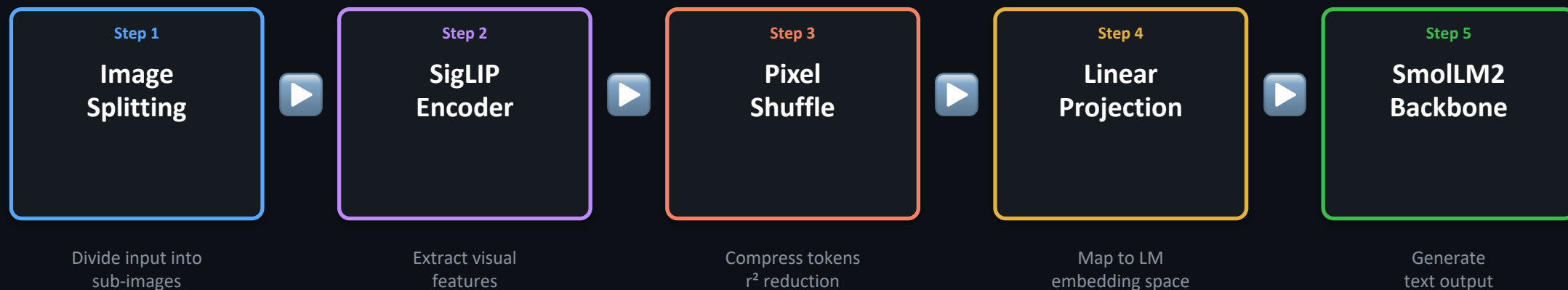
256M params

0.8 GB RAM

SmoVLM-256M surpasses the 300× larger Idefics-80B — architectural efficiency > raw scale

SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmolLM2 Backbone

End-to-end pipeline from image input to text generation



Three Critical Findings

1 Balanced Encoder-LM Compute

Smaller encoders outperform larger ones in compact VLMs

2 Extended Context Length

2K $\rightarrow$ 16K tokens unlocks higher image resolution

3 Aggressive Token Compression

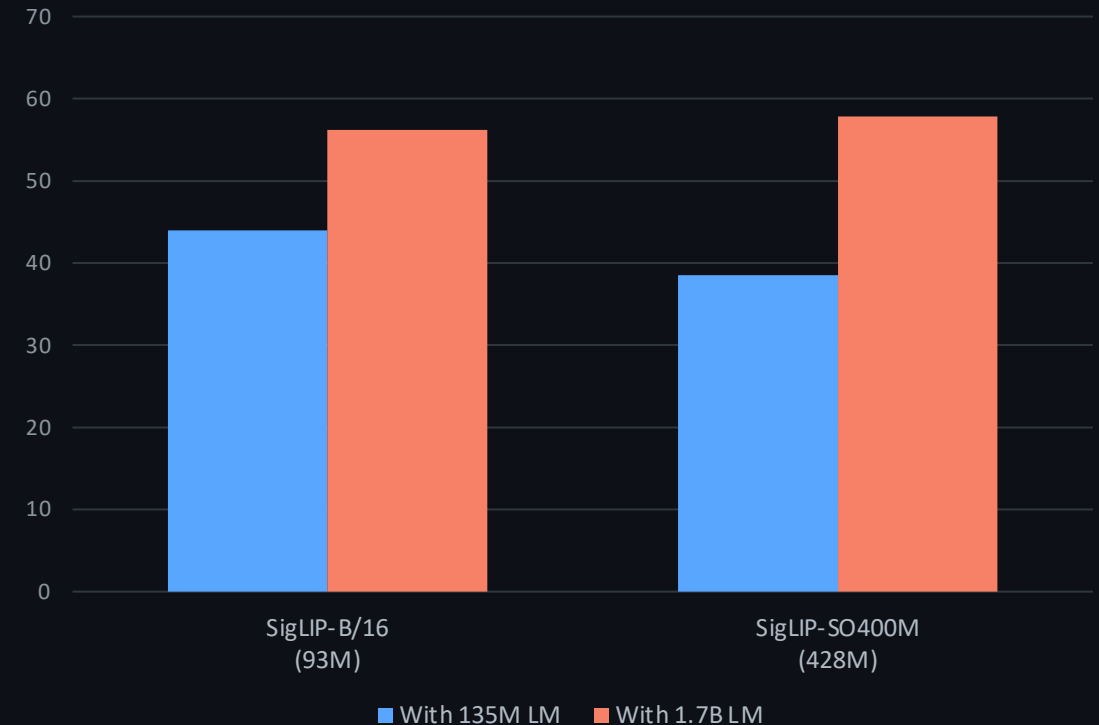
Pixel shuffle $r=4$ gives 16 $\times$ token reduction for small models

Video: frames sampled and processed through the same pipeline, concatenated with text tokens.

Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

Finding 1: Balanced encoder-LM parameter allocation is critical

- ▶ **93M SigLIP-B/16 outperforms 428M SigLIP-SO400M** when paired with a 135M LM
- ▶ **Larger encoder overhead only justified at 1.7B+ LM scale** 428M encoder adds 10% parameter overhead for small LMs
- ▶ **Key insight: allocate compute to the LM, not the encoder** compact VLMs need a stronger language backbone

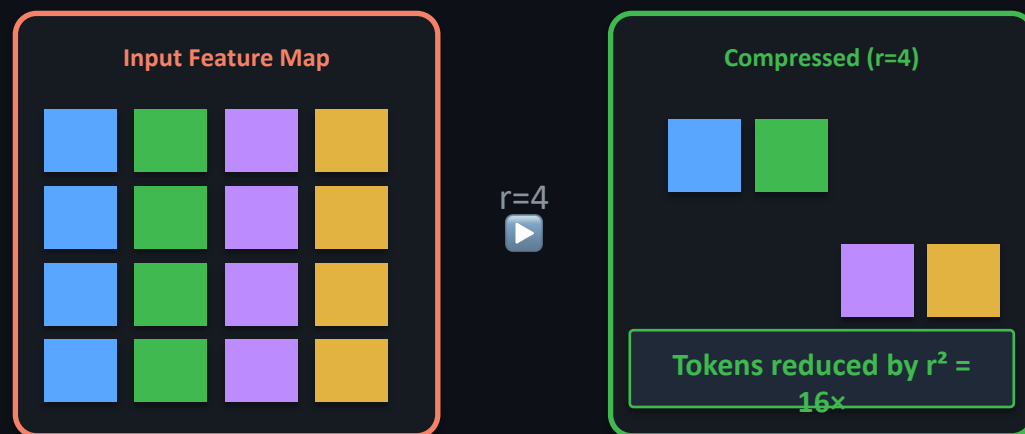


For compact VLMs, the smaller 93M encoder is the clear winner — the larger encoder's capacity is wasted when the LM can't exploit it

Aggressive Pixel Shuffle ($r=4$) Cuts Visual Tokens 16× for Small Models

Finding 3: Pixel shuffle rearranges spatial features into channels

Pixel Shuffle Mechanism (Space-to-Depth)



Shuffle Factor Comparison

$r = 2$ 4× token reduction

Better for larger models (2.2B)
Preserves more spatial detail

$r = 4$ 16× token reduction

Optimal for small models (256M, 500M)
Eases attention overhead & improves long-context

Small VLMs benefit most from aggressive compression — fewer tokens means less attention overhead and better long-context modeling

Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

RoPE base adjustment and mixed-context fine-tuning enable long sequences

Context Extension Strategy

- ▶ **RoPE base: 10K → 273K** Enables stable positional encoding at 16K length
- ▶ **Fine-tune on mixed long/short-context data** Prevents catastrophic forgetting of short-context tasks
- ▶ **2.2B model: 16K context length** Smaller variants (256M, 500M) use 8K context
- ▶ **Higher resolution via more tokens** Longer context accommodates more visual tokens from high-res images

Context Length by Model Size

SmoIVLM-256M



8K tokens

SmoIVLM-500M



8K tokens

SmoIVLM-2.2B



16K tokens

Impact on Performance

Significant performance improvement from 2K → 16K context:
higher image resolution processing without quality degradation

SmolVLM Achieves Competitive Performance at Sub-1GB to 4.9GB

12 benchmarks across single-image, multi-task, and video comprehension

RAM

Capability	Benchmark	SmolVLM 256M	SmolVLM 500M	SmolVLM 2.2B	Best Efficient OS
Single-Image	OCRBench	52.6%	61.0%	72.9%	54.7% (Molmo)
	AI2D	46.4%	59.2%	70.0%	71.0% (Molmo)
	ChartQA	55.6%	62.8%	68.7%	48.0% (Molmo)
	TextVQA	50.2%	60.2%	73.0%	61.5% (Molmo)
	DocVQA	58.3%	70.5%	80.0%	77.7% (Molmo)
	ScienceQA	73.8%	80.0%	89.6%	87.5% (Molmo)
Multi-task	MMMU	29.0%	33.7%	42.0%	33.9% (Molmo)
	MathVista	35.9%	40.1%	51.5%	37.6% (Molmo)
	MMStar	34.6%	38.3%	46.0%	43.1% (Molmo)
Video	Video-MME	33.7%	42.2%	52.1%	45.0% (InternVL2)
	MLVU	40.6%	47.3%	55.2%	48.2% (InternVL2)
AVERAGE (All Benchmarks)		44.0%	51.0%	59.8%	

GPU RAM (batch=1)

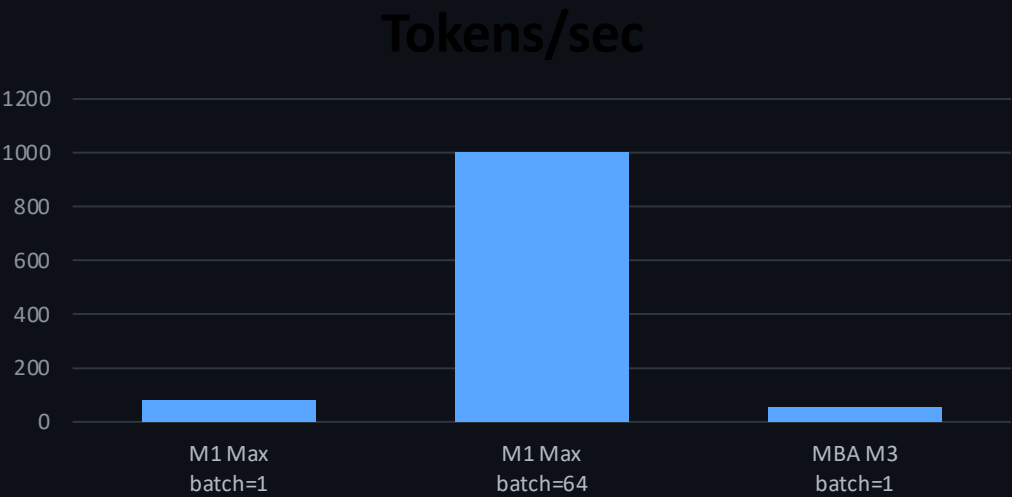
256M: 0.8 GB 500M: 1.2 GB 2.2B: 4.9 GB Molmo-A1B-7B: 27.7 GB

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon

HuggingSnap app — fully local, zero cloud dependency

SmolVLM-256M Throughput (tokens/second)

Device	Batch=1	Batch=64
Pro M1 Max 64GB	~80	~1000
MacBook Air M3 16GB	~55	—



Deployment Highlights

- ▶ **HuggingSnap iOS app** processes photos & videos entirely locally
- ▶ **Zero cloud dependency** all inference on-device for privacy
- ▶ **Throughput scales linearly with batch size** for the 256M model on high-end devices
- ▶ **Sub-1GB RAM at batch=1** enables smartphone and low-power edge deployment

Model Size vs. On-Device Feasibility

SmolVLM-256M	0.8 GB RAM	iPhone / edge
SmolVLM-500M	1.2 GB RAM	iPhone / edge
SmolVLM-2.2B	4.9 GB RAM	iPad Pro / laptop
Molmo-A1B-7B	27.7 GB RAM	Datacenter only

Key Takeaways: Design Principles for Efficient Multimodal Models

Actionable findings from SmolVLM for the on-device AI community

1

Balance Encoder-LM Compute

Smaller vision encoders (93M) outperform larger ones (428M) when paired with compact LMs — allocate parameters to the language backbone

2

Extend Context Length Aggressively

Scaling from 2K to 16K tokens via RoPE adjustment unlocks higher image resolution and significantly improves performance

3

Use Aggressive Token Compression for Small Models

Pixel shuffle $r=4$ provides 16× token reduction, reducing attention overhead and enabling long-context modeling in compact VLMs

4

Sub-1GB Inference Is Achievable

SmolVLM-256M runs at 0.8 GB RAM — surpassing the 300× larger LLaVA-80B, proving architectural efficiency beats raw scale

5

Fully Open-Source for Reproducibility

All weights, datasets, training code, and the HuggingSpan mobile app are released to promote on-device AI research

Architectural efficiency matters more than raw scale for on-device multimodal AI